

Equivalence class test case

(Task 3.2C)

Name : Ayush Indapure
Student ID : 224880003

Introduction

In this task, I performed equivalence class testing using JUnit for the DateUtil.java program provided in the base project. The main goal of this task was to test whether the system correctly increments and decrements dates by one day based on the given input values of day, month and year.

The valid input ranges are:

- Day: 1 to 31
- Month: 1 to 12
- Year: 1700 to 2024

Instead of testing every single possible date combination, equivalence class testing helps by dividing the inputs into logical groups where all values are expected to behave similarly. This saves time and makes testing more effective.

I also created an equivalence class test case for one of the internal webpages of Deakin University.

Equivalence Class Testing

Equivalence class testing is a black box testing technique where input data is divided into classes or groups. Each group contains values that are expected to produce the same result.

Instead of testing every value, we test one representative value from each class.

This method is useful because:

- it reduces number of test cases
- saves testing time
- improves test coverage
- helps identify invalid and valid input clearly

For DateUtil, the three variables are day, month and year.

Screenshot 1 – JUnit Test Results

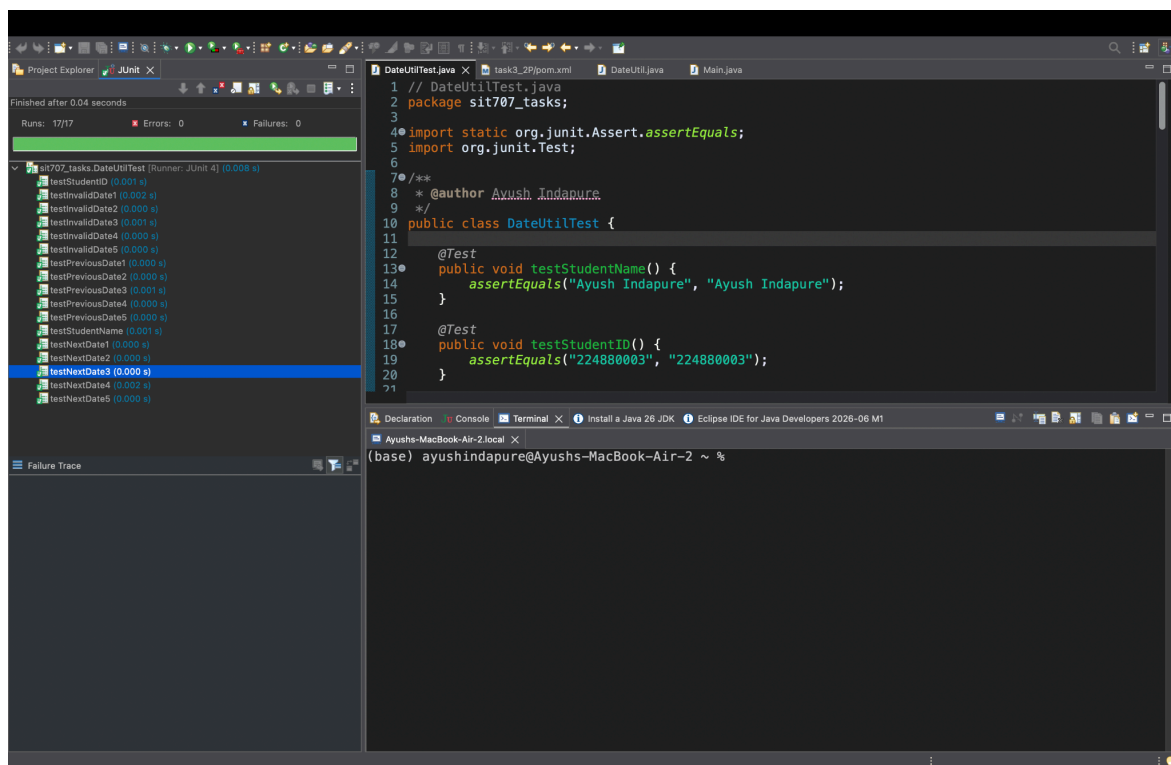
Example:

Runs: 17/17

Errors: 0

Failures: 0

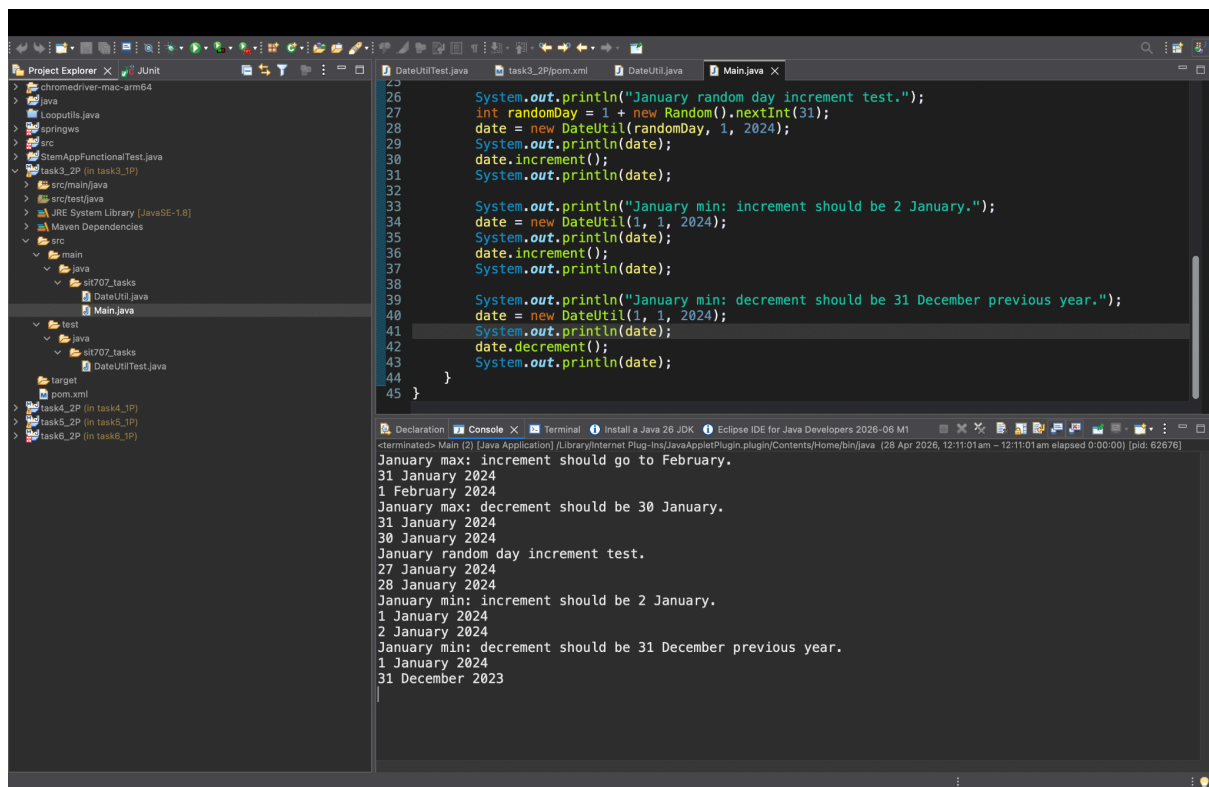
This proves that all test cases executed successfully.



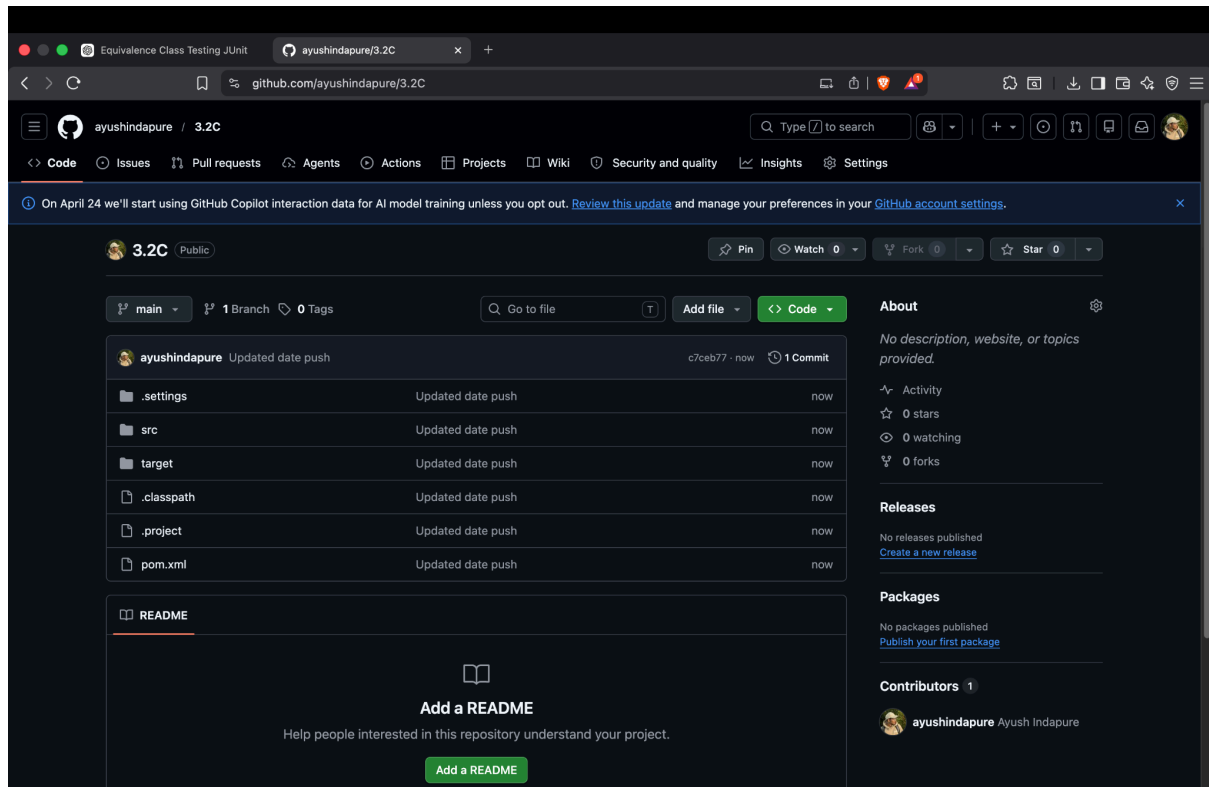
Screenshot 2 – Eclipse Console Output

This should show the output from running `Main.java`.

It helps demos that the increment and decrement functions are working correctly.



Screenshot 3 – GitHub Upload



Equivalence Classes for DateUtil

1. Day Classes

The day variable was divided into 4 classes:

- D1 = Day between 1 to 28
- D2 = Day = 29
- D3 = Day = 30
- D4 = Day = 31

This is important because February behaves differently and some months only allow 30 days.

2. Month Classes

The month variable was divided into 3 classes:

Months with 30 days

- April
- June
- September
- November

Months with 31 days

- January
- March
- May
- July
- August
- October
- December

February

- special because it depends on leap year

3. Year Classes

The year variable was divided into 2 classes:

Leap Year

Example: 2024

February has 29 days

Non-Leap Year

Example: 2023

February has 28 days

This is important because date increment and decrement changes for February.

Unit Test Case Creation using JUnit

JUnit is used for writing and running automated unit tests in Java.

I used JUnit 4 for this task and created test methods inside DateUtilTest.java.

Each test checks if the actual output matches the expected output using:

```
assertEquals()
```

For invalid dates, I used:

```
@Test(expected = RuntimeException.class)
```

This checks if the program throws an exception correctly.

Example:

```
new DateUtil(31, 6, 1994);
```

This is invalid because June only has 30 days.

Test Cases Implemented

I created tests for:

- previous date (decrement)
- next date (increment)
- leap year cases
- non leap year cases
- invalid input cases

Some examples are:

Increment Test

28 February 2024 → 29 February 2024

because 2024 is a leap year.

Increment Test

28 February 2023 → 1 March 2023

because 2023 is not a leap year.

Invalid Test

29 February 2023

should throw exception because 2023 is not leap year.

These tests help ensure the system behaves correctly for both normal and edge cases.

Equivalence Class Testing for Deakin Internal Website

Selected Website

I selected the login page of Deakin University OnTrack portal.

The feature tested is:

Student Login Page

where students enter:

- username / student email
- password

to access their study portal.

Equivalence Classes for OnTrack Login Page

Username Field

Valid Class

- correct Deakin student email
Example: ayush@deakin.edu.au

Invalid Class

- empty email
- wrong format
Example: ayushgmail.com
- non-existing account

Password Field

Valid Class

- correct password matching account

Invalid Class

- empty password
- wrong password
- too short password

Example Test Cases

Test Case	Input	Expected Result
Valid Login	Correct email + correct password	Login success
Empty Username	Blank email + password	Error message

Wrong Password	Correct email + wrong password	Login denied
Invalid Email Format	Wrong email format	Validation error
Empty Password	Correct email + blank password	Error message

Why Equivalence Class Testing is Suitable

This type of testing is suitable for login page because many users enter similar kinds of input.

For example, all invalid emails behave same way, so testing one or two values from that class is enough instead of testing hundreds.

It helps reduce repeated testing and improves efficiency.

It is also useful because login systems usually have strong validation rules, so grouping valid and invalid inputs makes testing much easier.

This method is very very easy but also very effective.

My DateUtilTest.java

// DateUtilTest.java

```
package sit707_tasks;
import static org.junit.Assert.assertEquals;
import org.junit.Test;
/**
 * @author Ayush Indapure
 */
public class DateUtilTest {
    @Test
    public void testStudentName() {
        assertEquals("Ayush Indapure", "Ayush Indapure");
    }
}
```

```

}
@Test
public void testStudentID() {
    assertEquals("224880003", "224880003");
}
// =====
// PREVIOUS DATE TESTS (decrement)
// =====
@Test
public void testPreviousDate1() {
    DateUtil date = new DateUtil(1, 6, 1994);
    date.decrement();
    assertEquals("31 May 1994", date.toString());
}
@Test
public void testPreviousDate2() {
    DateUtil date = new DateUtil(15, 6, 1994);
    date.decrement();
    assertEquals("14 June 1994", date.toString());
}
@Test
public void testPreviousDate3() {
    DateUtil date = new DateUtil(1, 1, 2024);
    date.decrement();
    assertEquals("31 December 2023", date.toString());
}
@Test
public void testPreviousDate4() {
    DateUtil date = new DateUtil(1, 3, 2024);
    date.decrement();
    assertEquals("29 February 2024", date.toString());
}

```

```

}
@Test
public void testPreviousDate5() {
    DateUtil date = new DateUtil(1, 3, 2023);
    date.decrement();
    assertEquals("28 February 2023", date.toString());
}

// =====
// NEXT DATE TESTS (increment)
// =====

@Test
public void testNextDate1() {
    DateUtil date = new DateUtil(1, 6, 1994);
    date.increment();
    assertEquals("2 June 1994", date.toString());
}

@Test
public void testNextDate2() {
    DateUtil date = new DateUtil(30, 6, 1994);
    date.increment();
    assertEquals("1 July 1994", date.toString());
}

@Test
public void testNextDate3() {
    DateUtil date = new DateUtil(31, 12, 2023);
    date.increment();
    assertEquals("1 January 2024", date.toString());
}

@Test
public void testNextDate4() {
    DateUtil date = new DateUtil(28, 2, 2024);

```

```

        date.increment();
        assertEquals("29 February 2024", date.toString());
    }

    @Test
    public void testNextDate5() {
        DateUtil date = new DateUtil(28, 2, 2023);
        date.increment();
        assertEquals("1 March 2023", date.toString());
    }

    // =====
    // INVALID DATE TESTS
    // =====

    @Test(expected = RuntimeException.class)
    public void testInvalidDate1() {
        new DateUtil(31, 6, 1994); // June has only 30 days
    }

    @Test(expected = RuntimeException.class)
    public void testInvalidDate2() {
        new DateUtil(29, 2, 2023); // 2023 is not leap year
    }

    @Test(expected = RuntimeException.class)
    public void testInvalidDate3() {
        new DateUtil(0, 5, 2024); // invalid day
    }

    @Test(expected = RuntimeException.class)
    public void testInvalidDate4() {
        new DateUtil(10, 13, 2024); // invalid month
    }

    @Test(expected = RuntimeException.class)
    public void testInvalidDate5() {
        new DateUtil(10, 5, 1699); // invalid year
    }

```

```
}  
}
```

Conclusion

In this task, I used equivalence class testing to test the DateUtil.java program and created JUnit test cases for increment and decrement operations.

I also applied the same testing concept to a real webpage from Deakin University which was the OnTrack login page.

This task helped me understand how software testing works in real projects and how JUnit makes testing easier and faster.

I also learned that testing is not only about finding bugs, but also making sure software works correctly for all important situations.

Overall this task improved my understanding of unit testing and software quality in a practical way.